

02393 Programming in C++

Module 6: Classes and Objects

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	LAB DAY + Guest lecture	N/A
9	05/11	Inheritance	19.1 - 19.3
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Object-Oriented Programming
- Classes in C++
- Abstract Data Types
- Lab

A recap from the last lecture

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays (statically and dynamically allocated), pointers, `enum`, `struct`, `vector`, `ifstream`, `ofstream`
- Expressions
 - some numeric and boolean operators and math functions
- Statements
 - `if`, `while`, `for`, `switch`

The “++” in C++

- So far, we have seen C programming with few elements of C++
 - string
 - cin/cout
 - references
 - vector
- C++ extends C with two key features:
 - Object-Oriented Programming (OOP) (today)
 - Templates for generic programming (next lecture)

Object-Oriented Programming

- We want to implement a library to handle bank accounts:
 - Each account will contain the following information
 - Name
 - Account number
 - Amount
 - Transferring money from one account to another

Object-Oriented Programming

- We can implement that library with struct and functions, however
 - The amount can be changed anywhere in our program
 - We want only the library code to change it
- Object-Oriented programming addresses this limitation

Object-Oriented Programming

- Procedural programming (what we did so far) focuses on actions:
 - Functions are the main elements of a program
 - Data exist primarily to support functions
- Object-Oriented programming focuses on data:
 - Data types and operations to manipulate them are grouped in classes
 - Classes are the main elements of a program
 - To represent data, classes are instantiated into objects

Classes in C++

- A class is similar to a struct, but its members can be:
 - Variables

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

Classes in C++

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)

Classes in C++

```
1 class BankAccount {  
2  
3     private  
4         int number;  
5         string owner;  
6         double balance;  
7  
8     public:  
9         BankAccount(int number, string owner, double balance);  
10        ~BankAccount();  
11  
12        double getBalance() const;  
13        int getNumber() const;  
14        const string getOwner() const;  
15        void print() const;  
16        bool transferMoney(BankAccount &to, double value);  
17        bool withdraw(double value);  
18        bool deposit(double value);  
19    };  
20  
21    double BankAccount::getBalance() const {  
22        return this->balance;  
23    }  
24  
25    ...
```

- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)
- Class members can be public or private
 - Users of a class can only access public members (data encapsulation)

Classes in C++

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)
- Class members can be public or private
 - Users of a class can only access public members (data encapsulation)
- Classes can have some special methods:
 - Constructor: called when an object is created, either statically, or dynamically using new

Classes in C++

```
1 class BankAccount {
2
3 private:
4     int number;
5     string owner;
6     double balance;
7
8 public:
9     BankAccount(int number, string owner, double balance);
10    ~BankAccount();
11
12    double getBalance() const;
13    int getNumber() const;
14    const string getOwner() const;
15    void print() const;
16    bool transferMoney(BankAccount &to, double value);
17    bool withdraw(double value);
18    bool deposit(double value);
19 };
20
21 double BankAccount::getBalance() const {
22     return this->balance;
23 }
24
25 ...
```

- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)
- Class members can be public or private
 - Users of a class can only access public members (data encapsulation)
- Classes can have some special methods:
 - Constructor: called when an object is created, either statically, or dynamically using new
 - Destructor: called when an object is destroyed, either statically by exiting a scope, or dynamically using delete

Classes in C++

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)
- Class members can be public or private
 - Users of a class can only access public members (data encapsulation)
- Classes can have some special methods:
 - Constructor: called when an object is created, either statically, or dynamically using new
 - Destructor: called when an object is destroyed, either statically by exiting a scope, or dynamically using delete

Classes in C++

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

- A class consists in:
 - Class definition:
 - Attributes
 - Method prototypes
 - Can be moved to a library header file

Classes in C++

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

- A class consists in:
 - Class definition:
 - Attributes
 - Method prototypes
 - Can be moved to a library header file
 - Class implementation:
 - Methods implementation

Using classes in C++

```
1  BankAccount accA(1, "Alice", 1000);  
2  BankAccount accB(2, "Bob", 500);  
3  
4  accA.print();  
5  accB.print();  
6  
7  accA.transferMoney(accB, 10);  
8  accA.print();  
9  accB.print();
```

- Instantiating an object
 - The constructor is called

Using classes in C++

```
1 BankAccount accA(1, "Alice", 1000);  
2 BankAccount accB(2, "Bob", 500);  
3  
4 accA.print();  
5 accB.print();  
6  
7 accA.transferMoney(accB, 10);  
8 accA.print();  
9 accB.print();
```

- Instantiating an object
 - The constructor is called
- Invoking a method for an object

Using classes in C++

```
1  #include <string>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      string str1("Hello World");
7      string str2 = "Hello world";
8
9      cout << "str1 is " << (str1.empty() ? "" : "not ")
10         << "empty" << endl;
11
12     cout << "str2 is " << str2.length()
13         << " characters long" << endl;
14 }
```

- Instantiating an object
 - The constructor is called
- Invoking a method for an object
- We have already seen all this during lecture 5.
 - E.g., when using strings

Redefining assignment operator

```
1 BankAccount accA(1, "alice", 1000);
2 BankAccount accB(2, "Bob", 500);
3
4 accB = accA;
5
6
7 BankAccount & BankAccount::operator=(BankAccount &ba){
8     this->owner = ba.owner;
9     return *this;
10 }
```

- By default, when object a is assigned to b, the content of a's attributes are copied over b's attributes (memberwise copy)
 - If a member is a pointer, only the address will be copied!
- To implement a different behaviour, we can redefine the assignment operator
 - this is the object to the left of the assignment
 - The input parameter is the object to the right of the assignment
 - Both are called by reference

Abstract Data Types

- Use C++ encapsulation to write code that abstracts from implementation details
 - Specify allowed operations on an ADT, by making them public
 - Hide everything else, by making it private
 - Instances of an ADT can only be constructed and used via public operations
- Programs that use a well-designed ADT do not need to be changed when the ADT's (private) implementation details are changed
- To show this principle, we will implement our own vector class, with the same behavior as the STL one
- Other examples are available in `lecture06-live-coding.zip` archive:
 - Matrix
 - Dictionary/Map
 - Bag

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 6
 - Ask questions if something is unclear (including previous assignments)

DTU

